



DATA STRUCTURES AND ALGORITHM

23CSE203

LAB MANUAL

SUBMITTED BY:	
NAME	S.Mohitha
ROLL NO	AV.SC.U4CSE25039
YEAR/SEM/SECTION	2nd/3rd/CSE A
CREDITS	05

SUBMITTED TO:	
NAME	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR (SI.Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	PROGRAM	PAGE NO
1	WEEK1: 1(A). A rocket launch displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!". 1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power) where: □ p = principal growth factor □ n = number of years 1(C). An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. 1(D). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. 1(E). Write a recursive Python program to print the first n terms of Fibonacci series	
2	WEEK2: 2(A). Write a Python program to implement linear search. 2(B) i. Write a Python program to implement binary search with sorted array as an input. 2(B) ii. Write a Python program to implement binary search without sorted array as an input	
3	WEEK 3: 3(A). Write a python program to implement bubble sort where the input elements are taken from the user 3(B). Write a python program to implement insertion sort where the input elements are taken from the user 3(C). Write a python program to implement selection sort where the input elements are taken from the user	

--	--	--

WEEK – 1

1)AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion .write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):  
    for i in range(n,-1,-1):  
        if i==0:  
            print("launch")  
        else:  
            print(i)  
  
n=int(input("enter no. of seconds to launch:"))  
launch(n)
```

OUTPUT:

```
>>>  
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_1.py =====  
enter no. of seconds to launch:7  
7  
6  
5  
4  
3  
2  
1  
launch  
>>> |
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

Error	Cause	Solution
Indentation Error	Incorrect indentation	Align blocks properly
Name Error	Variable not defined	Define before use
Value Error	Non-integer input	Enter valid integer

2)AIM: banks calculate the growth of an investment using compound interest the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n (power), where p =principal growth factor, n =number of years.

PROGRAM:

```
def intrest(p,n):  
    if n==0:  
        return 1  
    else:  
        return p*intrest(p,n-1)  
n=int(input("enter no.of years:"))  
p=int(input("enter a number:"))  
res=intrest(p,n)  
print(res)
```

OUTPUT:

```
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_2.py =====  
enter no.of years:4  
enter a number:25  
390625  
>>>
```

EXPLANATION:

This program recursively calculates p^n by multiplying p until n becomes 0.

ERROR TABLE:

Error	Cause	Solution
Recursion error	Very large value of n	Reduce input
Value Error	Non-integer input	Enter valid integer

3)AIM: An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID .write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(m):  
    key=m  
    for i in range(n-1):  
        if arr[i]==key:
```

```

        return i
n=int(input("enter no.of employees to add:"))
arr=[]
for i in range(n):
    a=int(input("enter a value:"))
    arr.append(a)
print(arr)
m=int(input("enter a element to search:"))
result=search(m)
if result==-1:
    print("element is not found")
else:
    print("employee id is stored at the index:",result)

```

OUTPUT:

```

>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_3.py =====
enter no.of employees to add:4
enter a value:11
enter a value:12
enter a value:16
enter a value:17
[11, 12, 16, 17]
enter a element to search:12
employee id is stored at the index: 1

```

EXPLANATION:

This program searches for an employee ID in a list and returns its index if found.

ERROR TABLE:

Error	Cause	Solution
Syntax error	Wrong quotes/brackets	Correct syntax
Logic error	Returns None instead of -1	Return -1 when not found

4)AIM: In a logistics company the number of possible ways to arrange n different parcels is calculated using the factorial formula($n!$). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def fact(n):  
    fact=1  
    for i in range(1,n+1):  
        fact=fact*i  
    return fact  
n=int(input("enter a number to find its factorial:"))  
result=fact(n)  
print(result)
```

OUTPUT:

```
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_4.py =====  
enter a number to find its factorial:12  
479001600  
>>>
```

EXPLANATION:

This program calculates the factorial of a number using iteration (not recursion).

ERROR TABLE:

Error	Cause	Solution
Overflow/large input	Large input value	Give smaller input
Value error	Invalid input	Even integer

5)AIM: write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fib(n):  
    if n==0:  
        return 0  
    elif n==1:  
        return 1  
    else:  
        return fib(n-1)+fib(n-2)  
n=int(input("enter no of terms:"))
```

```
if n<=-1:
    print("invalid input")
else:
    for i in range(n):
        print(fib(i),end=" ")
```

OUTPUT:

```
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_5.py =====
enter no of terms:5
0 1 1 2 3
>>>
```

EXPLANATION:

The program uses recursion to generate Fibonacci numbers. $\text{fib}(0)=0$ and $\text{fib}(1)=1$ are the base cases. For every other term, $\text{fib}(n)=\text{fib}(n-1) + \text{fib}(n-2)$. The loop prints the first n terms.

ERROR TABLE:

Error	Cause	Solution
Recursion error	Large value of n	Use iteration
Invalid input	Negative value of n	Validate input
Slow execution	Repeated recursive calls	Use dynamic programming

WEEK – 2

2(A). AIM: Write a Python program to implement linear search.

PROGRAM:

```
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search:"))
result = linear_search(arr,key)
if result!=1:
    print("Element found at index",result)
else:
    print("Element not found")
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2A.py =====
Enter the number of elements:4
Enter the elements:
11
14
9
23
Enter the element to search:23
Element found at index 3
>>> |
```

EXPLANATION:

The linear_search() function checks each element of the list one by one and compares

it with the given key.

If the key is found, its index is returned; otherwise, the function returns -1.

The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S NO	ERRORS	RECTIFICATION
	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2(B) i.AIM: Write a Python program to implement binary search with sorted array as an input.

PROGRAM:

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1
```

```
n = int(input("Enter the number of elements:"))  
arr = []
```

```
print("Enter the elements:")  
for i in range(n):  
    arr.append(int(input()))
```

```
key = int(input("Enter the element to search:"))
```

```
result = linear_search(arr, key)
```

```
if result != -1:
```

```
    print("Element found at index", result)
```

```
else:
```

```
    print("Element not found")
```

OUTPUT:

POSITIVE OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2B.py =====
Enter the number of elements:6
Enter the elements:
11
23
35
45
56
67
Enter the element to search:23
Element found at index 1
```

NEGATIVE OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2B.py =====
Enter the number of elements:6
Enter the elements:
11
23
35
45
56
67
Enter the element to search:87
Element not found
```

EXPLANATION:

1. The program takes a sorted array as input and searches for the required element using

binary search.

13

2. It checks the middle element and eliminates either the left or right half based on the

comparison.

3. If the element is found, its position is displayed; otherwise, it prints "Element not

found.”

S.NO	ERRORS	Rectification
1.	Binary search may give an incorrect result if the input array is unsorted.	Check whether the array is sorted and use arr.sort() before calling binary_search().
2.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly	Initialize low = 0, high = len(arr) - 1 and update them as low = mid + 1 or high = mid - 1.

2(B) ii.AIM: Write a Python program to implement binary search without sorted array as an input

PROGRAM:

```
def binary_search(arr,target):
    arr.sort()
    left = 0
    right = len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
```

```
target = int(input("Enter the element to search:"))
index = binary_search(arr,target)
print("Sorted array:",arr)
if index != -1:
    print(f"Element found at index {index} in the sorted array.")
else:
    print("Element not found.")
```

OUTPUT:

POSITIVE OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2Bii.py =====
Enter the number of elements:5
Enter the elements:
23
67
98
37
18
Enter the element to search:37
Sorted array: [18, 23, 37, 67, 98]
Element found at index 2 in the sorted array.
```

NEGATIVE OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2Bii.py =====
Enter the number of elements:5
Enter the elements:
23
67
98
37
18
Enter the element to search:64
Sorted array: [18, 23, 37, 67, 98]
Element not found.
```

EXPLANATION:

1. The program takes an unsorted array as input and first sorts it using arr.sort().
2. It then uses binary search by checking the middle element and reducing the search range.
3. If the element is found, its position is displayed; otherwise, it prints "Element not found."

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1.	Logical Error	Use a found variable and print "Element not found" if it remains False.
2.	Logical Error	Use $\text{mid} = (\text{low} + \text{high}) // 2$ to calculate the middle index.

WEEK-3

3A.

AIM: Write a python program to implement bubble sort where the input elements are taken from the user

PROGRAM:

```
n = int(input("Enter the number of elements: "))
```

```
arr = []
```

```
for i in range(n):
```

```
    element = int(input(f"Enter element {i + 1}: "))
```

```
    arr.append(element)
```

```
for i in range(n - 1):
```

```
    for j in range(n - 1 - i):
```

```
        if arr[j] > arr[j + 1]:
```

```
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
print("Sorted elements:", arr)
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab3_1.py =====
Enter the number of elements: 4
Enter element 1: 12
Enter element 2: 45
Enter element 3: 23
Enter element 4: 76
Sorted elements: [12, 23, 45, 76]
>>>
```

EXPLANATION:

1. The program takes the number of elements and their values from the user and stores

them in an array.

2. Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.

3. After all passes, the array is sorted in ascending order and displayed.

ERROR TABLE:

S.NO	ERRORS	RECTIFICATION
1	Name Error	Initialize arr = [] before taking input
2	Value Error	Enter only integer values

3(B).AIM:Write a python program to implement insertion sort where the input elements are

taken from the user

PROGRAM:

```
n = int(input("Enter the number of elements: "))
```

```
arr = []
```

```
for i in range(n):
```

```
    element = int(input(f"Enter element {i + 1}: "))
```

```
    arr.append(element)
```

```
for i in range(1, n):
```

```
    key = arr[i]
```

```
    j = i - 1
```

```
    while j >= 0 and arr[j] > key:
```

```
        arr[j + 1] = arr[j]
```

```
        j -= 1
```

```
    arr[j + 1] = key
```

```
print("Sorted elements:", arr)
```

OUTPUT:

```

===== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_2.py =====
Enter the number of elements: 5
Enter element 1: 12
Enter element 2: 32
Enter element 3: 45
Enter element 4: 21
Enter element 5: 97
Sorted elements: [12, 21, 32, 45, 97]
>>>

```

EXPLANATION:

- 1.The program takes the number of elements and their values from the user and stores them in an array.
- 2.Insertion Sort selects each element and inserts it into its correct position among the previously sorted elements.
3. Finally, the sorted array is displayed in ascending order.

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1	SyntaxError	Use insertion_sort with an underscore.
2	Name Error	Call insertion_sort(arr) correctly.

3(C).AIM:Write a python program to implement selection sort where the input elements are

taken from the user

PROGRAM:

```

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2

        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

```

```
i = 0
```

```
j = 0
```

```
k = 0
```

```
while i < len(left) and j < len(right):
```

```
    if left[i] < right[j]:
```

```
        arr[k] = left[i]
```

```
        i += 1
```

```
    else:
```

```
        arr[k] = right[j]
```

```
        j += 1
```

```
    k += 1
```

```
while i < len(left):
```

```
    arr[k] = left[i]
```

```
    i += 1
```

```
    k += 1
```

```
while j < len(right):
```

```
    arr[k] = right[j]
```

```
    j += 1
```

```
    k += 1
```

```
n = int(input("Enter the number of elements: "))
```

```
arr = []
```

```
for i in range(n):
```

```
    element = int(input(f"Enter element {i + 1}: "))
```

```
arr.append(element)
```

```
merge_sort(arr)
```

```
print("Sorted elements:", arr)
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_3.py =====
Enter the number of elements: 4
Enter element 1: 23
Enter element 2: 12
Enter element 3: 76
Enter element 4: 34
Sorted elements: [12, 23, 34, 76]
>>>
```

EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.
2. Selection Sort finds the smallest element and swaps it with the first unsorted element.
3. This process continues until the array is completely sorted in ascending order.

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1	NameError	Initialize arr = [] before taking input.
2	Value Error	Enter only integer values

3(d).AIM:Write a python program to implement quick sort where the input elements are

taken from the user

PROGRAM:

```
def quick_sort(arr):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    pivot = arr[-1]
```

```

left = []
right = []

for i in arr[:-1]:
    if i < pivot:
        left.append(i)
    else:
        right.append(i)

return quick_sort(left) + [pivot] + quick_sort(right)

n = int(input("Enter the number of elements:"))

arr = []

for i in range(n):
    element = int(input(f"Enter element{i + 1}:"))
    arr.append(element)

arr = quick_sort(arr)

print("Sorted elements:", arr)

```

OUTPUT:

```

>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_4.py =====
Enter the number of elements:4
Enter element1:23
Enter element2:12
Enter element3:65
Enter element4:87
Sorted elements: [12, 23, 65, 87]
>>>

```

3E.AIM:Write a python program to implement selection sort where the input elements are taken from the user

PROGRAM:

```
n = int(input("Enter the number of elements: "))
```

```
arr = []
```

```
for i in range(n):
```

```
    element = int(input(f"Enter element {i + 1}: "))
```

```
    arr.append(element)
```

```
for i in range(n - 1):
```

```
    min_index = i
```

```
    for j in range(i + 1, n):
```

```
        if arr[j] < arr[min_index]:
```

```
            min_index = j
```

```
    arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
print("Sorted elements:", arr)
```

```
>>>
```

```
===== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_5.py =====
```

```
Enter the number of elements: 6
```

```
Enter element 1: 98
```

```
Enter element 2: 68
```

```
Enter element 3: 45
```

```
Enter element 4: 65
```

```
Enter element 5: 23
```

```
Enter element 6: 16
```

```
Sorted elements: [16, 23, 45, 65, 68, 98]
```

```
>>>
```

EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.

2. Selection Sort finds the smallest element and swaps it with the first unsorted element.

3. This process continues until the array is completely sorted in ascending order.

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1.	TypeError	Convert input values using <code>int(input())</code> ..
2.	Logic Error	Update <code>min_index</code> correctly and swap <code>arr[i]</code> with <code>arr[min_index]</code> .

WEEK – 4

4A. AIM: Write a program to implement a single linked list which would perform the below operations, by taking an input from the user.

- a)create a linked list
- b)insert at beginning
- c)insert at end
- d)insert at a specified index
- e)delete by value
- f)delete 1st node
- g)delete last node

- h)count no.of nodes
- i)display or traverse
- j)search for a number
- k)exit

Program:

```
class Node:
```

```
    def __init__(self, data):  
        self.data = data  
        self.next = None
```

```
class LinkedList:
```

```
    def __init__(self):  
        self.head = None
```

```
    def create(self):  
        n = int(input("Enter number of elements: "))
```

```
        for i in range(n):  
            data = int(input("Enter data: "))  
            new = Node(data)
```

```
            if self.head is None:  
                self.head = new
```

```
            else:  
                temp = self.head  
                while temp.next:  
                    temp = temp.next  
                temp.next = new
```

```
def insert_begin(self, data):
```

```
    new = Node(data)
```

```
    new.next = self.head
```

```
    self.head = new
```

```
def insert_end(self, data):
```

```
    new = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next:
```

```
            temp = temp.next
```

```
        temp.next = new
```

```
def insert_index(self, index, data):
```

```
    if index < 0:
```

```
        print("Invalid index")
```

```
        return
```

```
    if index == 0:
```

```
        self.insert_begin(data)
```

```
        return
```

```
    temp = self.head
```

```
    for i in range(index - 1):
```

```
        if temp is None:
```

```
            print("Invalid index")
```

```
        return  
    temp = temp.next
```

```
if temp is None:  
    print("Invalid index")  
    return
```

```
new = Node(data)  
new.next = temp.next  
temp.next = new
```

```
def delete_value(self, data):  
    if self.head is None:  
        print("Linked list is empty")  
        return
```

```
    if self.head.data == data:  
        self.head = self.head.next  
        return
```

```
    temp = self.head
```

```
    while temp.next:  
        if temp.next.data == data:  
            temp.next = temp.next.next  
            return  
        temp = temp.next
```

```
    print("Value not found")
```

```
def delete_first(self):  
    if self.head is None:  
        print("Linked list is empty")  
    else:  
        self.head = self.head.next
```

```
def delete_last(self):  
    if self.head is None:  
        print("Linked list is empty")  
        return
```

```
    if self.head.next is None:  
        self.head = None  
        return
```

```
    temp = self.head
```

```
    while temp.next.next:  
        temp = temp.next
```

```
    temp.next = None
```

```
def count(self):  
    count = 0  
    temp = self.head  
  
    while temp:  
        count = count + 1  
        temp = temp.next
```

```
print("Number of nodes:", count)
```

```
def display(self):
```

```
    temp = self.head
```

```
    while temp:
```

```
        print(temp.data, end=" -> ")
```

```
        temp = temp.next
```

```
    print("None")
```

```
l = LinkedList()
```

```
l.create()
```

```
print("Linked list:", end=" ")
```

```
l.display()
```

```
data = int(input("Enter data to insert at beginning: "))
```

```
l.insert_begin(data)
```

```
print("After insertion:", end=" ")
```

```
l.display()
```

```
data = int(input("Enter data to insert at end: "))
```

```
l.insert_end(data)
```

```
print("After insertion:", end=" ")
```

```
l.display()
```

```
index = int(input("Enter index: "))
```

```
data = int(input("Enter data: "))
```

```

l.insert_index(index, data)
print("After insertion:", end=" ")
l.display()

data = int(input("Enter value to delete: "))
l.delete_value(data)
print("After deletion:", end=" ")
l.display()

l.delete_first()
print("After deleting first node:", end=" ")
l.display()

l.delete_last()
print("After deleting last node:", end=" ")
l.display()

l.count()

```

OUTPUT:

```

>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab4_1.py =====
Enter number of elements: 5
Enter data: 22
Enter data: 12
Enter data: 54
Enter data: 87
Enter data: 67
Linked list: 22 -> 12 -> 54 -> 87 -> 67 -> None
Enter data to insert at beginning: 12
After insertion: 12 -> 22 -> 12 -> 54 -> 87 -> 67 -> None
Enter data to insert at end: 54
After insertion: 12 -> 22 -> 12 -> 54 -> 87 -> 67 -> 54 -> None
Enter index: 3
Enter data: 67
After insertion: 12 -> 22 -> 12 -> 67 -> 54 -> 87 -> 67 -> 54 -> None
Enter value to delete: 22
After deletion: 12 -> 12 -> 67 -> 54 -> 87 -> 67 -> 54 -> None
After deleting first node: 12 -> 67 -> 54 -> 87 -> 67 -> 54 -> None
After deleting last node: 12 -> 67 -> 54 -> 87 -> 67 -> None
Number of nodes: 5
>>> |

```

EXPLANATION:

The LinkedList program creates and performs different operations on a singly linked list.

The create() function creates the linked list, while insert_begin(), insert_end(), and insert_index() insert new nodes at the beginning, end, and specified index.

The delete_value(), delete_first(), and delete_last() functions delete nodes from the list. The count() function counts the number of nodes, and the display() function traverses and displays all the elements.

In a singly linked list, each node contains two parts: data and next. The data stores the value, while next stores the reference to the next node. The next pointer of the last node is always None.

S NO ERRORS

RECTIFICATION

- | | | |
|---|---|---|
| 1 | ValueError occurs if a non-integer value is entered when int(input()) is used. | Enter only integer values for the number of elements, index, and node data. |
| 2 | IndentationError may occur if statements inside classes, functions, if, else, while, or for blocks are not properly indented. | Maintain proper indentation throughout the program. |
| 3 | NameError may occur if variables such as temp, new, or data are misspelled or used before declaration. | Check variable names and initialize them before use. |

4B.AIM: Write a program to implement a double linked list which would perform the below operations, by taking an input from the user.

PROGRAM:

```
class Node:
```

```
    def __init__(self, data):  
        self.prev = None  
        self.data = data  
        self.next = None
```

```
class DoublyLinkedList:
```

```
def __init__(self):
    self.head = None

# 1. Create linked list
def create(self):
    n = int(input("Enter number of elements: "))

    for i in range(n):
        data = int(input("Enter data: "))
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new
            new.prev = temp

# 2. Insert at beginning
def insert_begin(self, data):
    new = Node(data)

    new.next = self.head

    if self.head is not None:
        self.head.prev = new
```

```
self.head = new
```

3. Insert at end

```
def insert_end(self, data):
```

```
    new = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next:
```

```
            temp = temp.next
```

```
        temp.next = new
```

```
        new.prev = temp
```

4. Insert at specific index

```
def insert_index(self, index, data):
```

```
    if index < 0:
```

```
        print("Invalid index")
```

```
        return
```

```
    if index == 0:
```

```
        self.insert_begin(data)
```

```
        return
```

```
    temp = self.head
```

```
for i in range(index - 1):  
    if temp is None:  
        print("Invalid index")  
        return  
    temp = temp.next
```

```
if temp is None:  
    print("Invalid index")  
    return
```

```
new = Node(data)
```

```
new.next = temp.next  
new.prev = temp
```

```
if temp.next is not None:  
    temp.next.prev = new
```

```
temp.next = new
```

5. Delete by value

```
def delete_value(self, data):  
    if self.head is None:  
        print("Linked list is empty")  
        return
```

```
temp = self.head
```

```
while temp:  
    if temp.data == data:
```

```
    if temp.prev is not None:
        temp.prev.next = temp.next
    else:
        self.head = temp.next

    if temp.next is not None:
        temp.next.prev = temp.prev

    return
```

```
temp = temp.next
```

```
print("Value not found")
```

6. Delete first node

```
def delete_first(self):
    if self.head is None:
        print("Linked list is empty")
        return
```

```
self.head = self.head.next
```

```
if self.head is not None:
    self.head.prev = None
```

7. Delete last node

```
def delete_last(self):
    if self.head is None:
        print("Linked list is empty")
```

```
return
```

```
temp = self.head
```

```
while temp.next:
```

```
    temp = temp.next
```

```
if temp.prev is None:
```

```
    self.head = None
```

```
else:
```

```
    temp.prev.next = None
```

```
# 8. Count number of nodes
```

```
def count(self):
```

```
    count = 0
```

```
    temp = self.head
```

```
    while temp:
```

```
        count = count + 1
```

```
        temp = temp.next
```

```
    print("Number of nodes:", count)
```

```
# 9. Display / Traverse
```

```
def display(self):
```

```
    temp = self.head
```

```
    while temp:
```

```
        print(temp.data, end=" <-> ")
```

```
        temp = temp.next
```

```
print("None")
```

```
l = DoublyLinkedList()
```

```
l.create()
```

```
print("Linked list:", end=" ")
```

```
l.display()
```

```
data = int(input("Enter data to insert at beginning: "))
```

```
l.insert_begin(data)
```

```
print("After insertion:", end=" ")
```

```
l.display()
```

```
data = int(input("Enter data to insert at end: "))
```

```
l.insert_end(data)
```

```
print("After insertion:", end=" ")
```

```
l.display()
```

```
index = int(input("Enter index: "))
```

```
data = int(input("Enter data: "))
```

```
l.insert_index(index, data)
```

```
print("After insertion:", end=" ")
```

```
l.display()
```

```
data = int(input("Enter value to delete: "))
```

```
l.delete_value(data)
```

```
print("After deletion:", end=" ")
```

```
l.display()
```

`l.delete_first()`

`print("After deleting first node:", end=" ")`

`l.display()`

`l.delete_last()`

`print("After deleting last node:", end=" ")`

`l.display()`

`l.count()`

OUTPUT:

```
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab4_2.py =====
Enter number of elements: 6
Enter data: 34
Enter data: 98
Enter data: 56
Enter data: 15
Enter data: 72
Enter data: 45
Linked list: 34 <-> 98 <-> 56 <-> 15 <-> 72 <-> 45 <-> None
Enter data to insert at beginning: 56
After insertion: 56 <-> 34 <-> 98 <-> 56 <-> 15 <-> 72 <-> 45 <-> None
Enter data to insert at end: 72
After insertion: 56 <-> 34 <-> 98 <-> 56 <-> 15 <-> 72 <-> 45 <-> 72 <-> None
Enter index: 4
Enter data: 61
After insertion: 56 <-> 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> 45 <-> 72 <-> None
Enter value to delete: 45
After deletion: 56 <-> 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> 72 <-> None
After deleting first node: 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> 72 <-> None
After deleting last node: 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> None
Number of nodes: 6
>>>
```

EXPLANATION:

The DoublyLinkedList program creates and performs different operations on a doubly linked list.

The `create()` function creates the linked list, while `insert_begin()`, `insert_end()`, and `insert_index()` insert new nodes at the beginning, end, and specified index.

The `delete_value()`, `delete_first()`, and `delete_last()` functions delete nodes from the list. The `count()` function counts the number of nodes, and the `display()` function displays all the elements.

In a doubly linked list, each node contains **three parts: prev, data, and next**. The prev pointer points to the previous node, while the next pointer points to the next node.

S NO ERRORS

RECTIFICATION

- | | | |
|---|---|---|
| 1 | ValueError occurs if a non-integer value is entered when int(input()) is used. | Enter only integer values for the number of elements, index, and node data. |
| 2 | IndentationError may occur if statements inside classes, functions, if, else, while, or for blocks are not properly indented. | Maintain proper indentation throughout the program. |
| 3 | AttributeError may occur if an incorrect attribute such as data, prev, or next is used. | Make sure self.data, self.prev, and self.next are used correctly. |
| 4 | NameError may occur if variables such as temp, new, or data are misspelled or used before declaration. | Check variable names and initialize them before use. |

4C.AIM: Write a program to implement a circular linked list which would perform the below operations, by taking an input from the user.

PROGRAM:

```
class Node:
```

```
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class CircularLinkedList:
```

```
    def __init__(self):
        self.head = None
```

```
# 1. Create linked list
```

```
def create(self):
    n = int(input("Enter number of elements: "))

    for i in range(n):
```

```
data = int(input("Enter data: "))
```

```
new = Node(data)
```

```
if self.head is None:
```

```
    self.head = new
```

```
    new.next = self.head
```

```
else:
```

```
    temp = self.head
```

```
    while temp.next != self.head:
```

```
        temp = temp.next
```

```
    temp.next = new
```

```
    new.next = self.head
```

2. Insert at beginning

```
def insert_begin(self, data):
```

```
    new = Node(data)
```

```
if self.head is None:
```

```
    self.head = new
```

```
    new.next = self.head
```

```
else:
```

```
    temp = self.head
```

```
    while temp.next != self.head:
```

```
        temp = temp.next
```

```
    new.next = self.head
```

```
    self.head = new
```

```
temp.next = self.head
```

3. Insert at end

```
def insert_end(self, data):
```

```
    new = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new
```

```
        new.next = self.head
```

```
    else:
```

```
        temp = self.head
```

```
        while temp.next != self.head:
```

```
            temp = temp.next
```

```
        temp.next = new
```

```
        new.next = self.head
```

4. Insert at specific index

```
def insert_index(self, index, data):
```

```
    if index < 0:
```

```
        print("Invalid index")
```

```
        return
```

```
    if index == 0:
```

```
        self.insert_begin(data)
```

```
        return
```

```
    if self.head is None:
```

```
        print("Invalid index")
```

```
return
```

```
temp = self.head
```

```
for i in range(index - 1):
```

```
    temp = temp.next
```

```
if temp == self.head:
```

```
    print("Invalid index")
```

```
    return
```

```
new = Node(data)
```

```
new.next = temp.next
```

```
temp.next = new
```

```
# 5. Delete by value
```

```
def delete_value(self, data):
```

```
    if self.head is None:
```

```
        print("Linked list is empty")
```

```
        return
```

```
if self.head.data == data:
```

```
    self.delete_first()
```

```
    return
```

```
temp = self.head
```

```
while temp.next != self.head:
```

```
    if temp.next.data == data:
```

```
        temp.next = temp.next.next
```

```
        return
    temp = temp.next

    print("Value not found")
```

6. Delete first node

```
def delete_first(self):
    if self.head is None:
        print("Linked list is empty")
        return

    if self.head.next == self.head:
        self.head = None
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    self.head = self.head.next
    temp.next = self.head
```

7. Delete last node

```
def delete_last(self):
    if self.head is None:
        print("Linked list is empty")
        return

    if self.head.next == self.head:
```

```
self.head = None
```

```
return
```

```
temp = self.head
```

```
while temp.next.next != self.head:
```

```
    temp = temp.next
```

```
temp.next = self.head
```

```
# 8. Count number of nodes
```

```
def count(self):
```

```
    if self.head is None:
```

```
        print("Number of nodes: 0")
```

```
    return
```

```
count = 0
```

```
temp = self.head
```

```
while True:
```

```
    count = count + 1
```

```
    temp = temp.next
```

```
    if temp == self.head:
```

```
        break
```

```
print("Number of nodes:", count)
```

```
# 9. Display / Traverse
```

```
def display(self):
```

```
if self.head is None:  
    print("Linked list is empty")  
    return
```

```
temp = self.head
```

```
while True:  
    print(temp.data, end=" -> ")  
    temp = temp.next
```

```
if temp == self.head:  
    break
```

```
print("(back to head)")
```

```
l = CircularLinkedList()
```

```
l.create()  
print("Linked list:", end=" ")  
l.display()
```

```
data = int(input("Enter data to insert at beginning: "))  
l.insert_begin(data)  
print("After insertion:", end=" ")  
l.display()
```

```
data = int(input("Enter data to insert at end: "))  
l.insert_end(data)  
print("After insertion:", end=" ")
```

```
l.display()
```

```
index = int(input("Enter index: "))
```

```
data = int(input("Enter data: "))
```

```
l.insert_index(index, data)
```

```
print("After insertion:", end=" ")
```

```
l.display()
```

```
data = int(input("Enter value to delete: "))
```

```
l.delete_value(data)
```

```
print("After deletion:", end=" ")
```

```
l.display()
```

```
l.delete_first()
```

```
print("After deleting first node:", end=" ")
```

```
l.display()
```

```
l.delete_last()
```

```
print("After deleting last node:", end=" ")
```

```
l.display()
```

```
l.count()
```

OUTPUT:

```

>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab4_3.py =====
Enter number of elements: 5
Enter data: 31
Enter data: 98
Enter data: 47
Enter data: 86
Enter data: 34
Linked list: 31 -> 98 -> 47 -> 86 -> 34 -> (back to head)
Enter data to insert at beginning: 31
After insertion: 31 -> 31 -> 98 -> 47 -> 86 -> 34 -> (back to head)
Enter data to insert at end: 47
After insertion: 31 -> 31 -> 98 -> 47 -> 86 -> 34 -> 47 -> (back to head)
Enter index: 4
Enter data: 98
After insertion: 31 -> 31 -> 98 -> 47 -> 98 -> 86 -> 34 -> 47 -> (back to head)
Enter value to delete: 86
After deletion: 31 -> 31 -> 98 -> 47 -> 98 -> 34 -> 47 -> (back to head)
After deleting first node: 31 -> 98 -> 47 -> 98 -> 34 -> 47 -> (back to head)
After deleting last node: 31 -> 98 -> 47 -> 98 -> 34 -> (back to head)
Number of nodes: 5
>>>

```

EXPLANATION:

The CircularLinkedList program performs various operations such as creating, inserting, deleting, counting, and displaying elements in a circular linked list.

The create() function creates the circular linked list, while insert_begin(), insert_end(), and insert_index() insert new nodes at different positions.

The delete_value(), delete_first(), and delete_last() functions remove nodes from the list. The count() function counts the number of nodes, and display() traverses and displays all the elements.

In a circular linked list, the last node always points back to the first node (head).

ERROR TABLE:

S NO	ERRORS	RECTIFICATION
1	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements, index, and node data.
2	IndentationError may occur if the statements inside classes, functions, if, else, while, or for blocks are not properly indented.	Maintain proper indentation throughout the program.
3	AttributeError may occur if an incorrect attribute such as data or next is used.	Make sure self.data and self.next are used correctly.

5.AIM: Write a Python program to implement a Stack using an Array and a Linked List. Implement stack operations such as push, pop, peek, and display the stack elements.

a.Stack Using Array:

PROGRAM:

```
class StackArray:
```

```
    def __init__(self):
```

```
        self.stack = []
```

```
    # Push operation
```

```
    def push(self, data):
```

```
        self.stack.append(data)
```

```
        print(data, "pushed into stack")
```

```
    # Pop operation
```

```
    def pop(self):
```

```
        if not self.stack:
```

```
            print("Stack Underflow")
```

```
        else:
```

```
            print(self.stack.pop(), "popped from stack")
```

```
    # Peek operation
```

```
    def peek(self):
```

```
        if not self.stack:
```

```
            print("Stack is empty")
```

```
        else:
```

```
            print("Top element:", self.stack[-1])
```

```
    # Display operation
```

```
    def display(self):
```

```
        if not self.stack:
```

```
print("Stack is empty")
```

```
else:
```

```
print("Stack elements:", self.stack[::-1])
```

```
s = StackArray()
```

```
n = int(input("Enter number of elements: "))
```

```
for i in range(n):
```

```
    data = int(input("Enter data: "))
```

```
    s.push(data)
```

```
print("Stack:")
```

```
s.display()
```

```
s.peak()
```

```
data = int(input("Enter data to push: "))
```

```
s.push(data)
```

```
print("After push:")
```

```
s.display()
```

```
s.pop()
```

```
print("After pop:")
```

```
s.display()
```

```
s.peak()
```

OUTPUT:

```
>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab5_2.py =====
Enter number of elements: 4
Enter data: 17
17 pushed into stack
Enter data: 12
12 pushed into stack
Enter data: 26
26 pushed into stack
Enter data: 16
16 pushed into stack
Stack:
Stack elements: [16, 26, 12, 17]
Top element: 16
Enter data to push: 6
6 pushed into stack
After push:
Stack elements: [6, 16, 26, 12, 17]
6 popped from stack
After pop:
Stack elements: [16, 26, 12, 17]
Top element: 16
>>>
```

b.Stack Using Linked List:

PROGRAM:

class Node:

```
    def __init__(self, data):
        self.data = data
        self.next = None
```

class StackLinkedList:

```
    def __init__(self):
        self.top = None
```

Push operation

```
def push(self, data):
    new = Node(data)
    new.next = self.top
    self.top = new
    print(data, "pushed into stack")
```

Pop operation

```
def pop(self):
    if self.top is None:
```

```
    print("Stack Underflow")
```

```
else:
```

```
    print(self.top.data, "popped from stack")
```

```
    self.top = self.top.next
```

```
# Peek operation
```

```
def peek(self):
```

```
    if self.top is None:
```

```
        print("Stack is empty")
```

```
    else:
```

```
        print("Top element:", self.top.data)
```

```
# Display operation
```

```
def display(self):
```

```
    if self.top is None:
```

```
        print("Stack is empty")
```

```
        return
```

```
    temp = self.top
```

```
    print("Stack elements:", end=" ")
```

```
    while temp:
```

```
        print(temp.data, end=" ")
```

```
        temp = temp.next
```

```
    print()
```

```
s = StackLinkedList()
```

```
n = int(input("Enter number of elements: "))
```

```
for i in range(n):
```

```
    data = int(input("Enter data: "))
```

```
    s.push(data)
```

```
print("Stack:")
```

```
s.display()
```

```
s.peek()
```

```
data = int(input("Enter data to push: "))
```

```
s.push(data)
```

```
print("After push:")
```

```
s.display()
```

```
s.pop()
```

```
print("After pop:")
```

```
s.display()
```

```
s.peek()
```

OUTPUT:

```

>>>
===== RESTART: C:/Users/mohit/OneDrive/Desktop/lab5_1.py =====
Enter number of elements: 5
Enter data: 11
11 pushed into stack
Enter data: 7
7 pushed into stack
Enter data: 15
15 pushed into stack
Enter data: 9
9 pushed into stack
Enter data: 12
12 pushed into stack
Stack:
Stack elements: 12 9 15 7 11
Top element: 12
Enter data to push: 4
4 pushed into stack
After push:
Stack elements: 4 12 9 15 7 11
4 popped from stack
After pop:
Stack elements: 12 9 15 7 11
Top element: 12
>>>

```

EXPLANATION:

The program implements a Stack using both an Array and a Linked List.

A stack follows the LIFO (Last In, First Out) principle, where the element inserted last is removed first.

The push() operation adds an element to the top of the stack, while the pop() operation removes the top element. The peek() operation displays the top element without removing it. The display() operation displays all the elements present in the stack.

In the array implementation, Python's list is used to store stack elements. In the linked-list implementation, nodes are created dynamically and the top pointer represents the top of the stack.

S NO ERRORS

RECTIFICATION

- | | | |
|---|---|--|
| 1 | Stack Underflow occurs when pop() is performed on an empty stack. | Check whether the stack is empty before performing pop(). |
| 2 | Stack is empty occurs when peek() or display() is performed without any elements. | Push elements into the stack before performing these operations. |
| 3 | Incorrect top element may be displayed if the top position is not updated properly. | Update the top pointer or array position correctly after every push and pop. |

